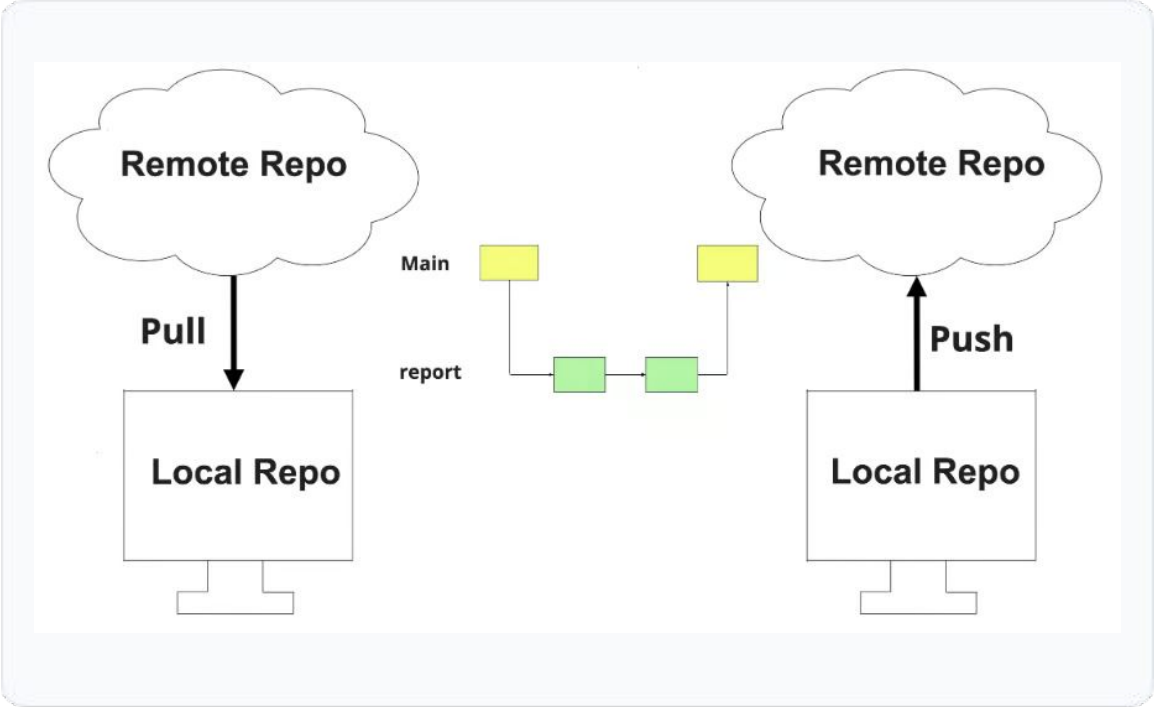
Aula 3:
GitHub & Colaboração

Repositórios Remotos

São versões do projeto que estão hospedadas em outro local (como a Internet)

```
git clone
https://github.com/user/repo.git
```

- **Nuvem:** GitHub, GitLab, BitBucket, etc.
- **Origin:** Nome padrão que o Git dá ao servidor remoto após clonar.
- **Protocolos autenticação:** HTTPS vs SSH.



Gerenciando Remotos

Listando Conexões Ativas

O parâmetro -v exibe as URLs vinculadas ao repositório local (para *fetch* e *push*).

```
$ git remote -v
origin https://github… (fetch)
origin https://github... (push)
john http://john-laptop.org/test…
```

Adicionando Novos Repos Remotos

`git remote add <shortname> <url>`

Um repositório local pode sincronizar com múltiplos remotos diferentes se necessário.

```
$ git remote
add john http://john-laptop.org/test…
```

Sincronizando Código

Fetch

Baixa dados do remoto (commits, branches), mas **não altera** seus arquivos de trabalho localmente.

```
git fetch origin
```

Pull

Fetch + Merge automático. Baixa os dados e tenta atualizar sua branch atual instantaneamente.

```
git pull origin master
```

Push

Envia seus commits locais finalizados para o servidor (exige merge fast-forward).

```
git push origin master
```

Desfazendo Mudanças: O comando Reset

O comando git reset move o HEAD da branch atual para um commit específico no passado. Ele atua em três níveis diferentes:

--soft

O modo mais brando. **Não toca** na Staging Area (index) nem nos seus arquivos de trabalho.

--mixed (Padrão)

Faz reset na Staging Area, mas **mantém** as alterações nos arquivos de trabalho locais.

--hard

Apaga tudo! Faz reset da Staging Area e descarta permanentemente alterações nos arquivos de trabalho.

Reset na Prática (Local)

Alterações Locais (Unstaged)

Descartando modificações antes mesmo de adicioná-las à Staging Area.

```
# Desfaz alterações em UM arquivo
$ git checkout -- README.md

# Descarta TODAS as alterações locais
$ git reset --hard
```

Alterações na Staging (Uncommited)

Tirando arquivos da Stating (mantendo as modificações seguras no disco/local).

```
# Remove UM arquivo ds Stating
$ git reset HEAD

# Remove TODOS os ficheiros da Stating
$ git reset
```

Navegando no Histórico

Os símbolos ~ (tilde) e ^ (caret) apontam para posições relativas a um commit, evitando o uso de hashes extensos.

- **HEAD^ :** Refere-se exatamente ao commit anterior ao HEAD.
- **HEAD^^ :** Refere-se a dois commits atrás.
- **HEAD~2 :** Também refere-se a dois commits atrás (mais limpo para grandes saltos, ex: HEAD~5).

```
# Apagar o último commit local
# (que ainda não foi enviado/pushed)

$ git reset --hard HEAD^
```

Revertendo Mudanças Públicas

Deve-se tentar NUNCA reescrever o histórico público.

Se um commit com erro já foi enviado (**pushed**) para o repositório remoto, a aposta segura para desfazer algo e evitar problemas de colaboração é usar o **revert**.

```
git revert
```

Isto não apaga o passado. Em vez disso, introduz um **novo commit** que anula exatamente as modificações do commit defeituoso, preservando a integridade.

Fluxos de Trabalho (Git Flow)

Não existe apenas uma maneira de usar o Git. O importante é a consistência em adotar um padrão (Exemplo Clássico):

- **Master / Main:** O ambiente sagrado. Apenas código de produção estável, revisado e finalizado chega aqui.
- **Develop:** A branch principal de desenvolvimento. É aqui que a integração de todas as novas funcionalidades acontece.
- **Feature Branches:** Ramificações temporárias (criadas a partir da *Develop*) isoladas para o desenvolvimento de cada nova funcionalidade específica.
- **Hotfix:** Branches criadas exclusivamente a partir da *Main* para resolver falhas críticas urgentes em produção, fazendo merge de volta para a Main e Develop imediatamente.

Referências

- Slides prof. André Restivo https://pages.up.pt/~up353972/slides/?s=git#2
- Toda a documentação do Git para quando quiserem consultar e aprofundar os conceitos vistos no Minicurso: https://git-scm.com/book/ms/v2/Getting-Started-About-Version-Control
- DataCamp: https://www.datacamp.com/pt/tutorial/git-remote